```
public:
    thread_pool():
        done(false),joiner(threads)
    {
        unsigned const thread_count=std::thread::hardware_concurrency(); ❽

        try
        {
            for(unsigned i=0;i<thread_count;++i)
            {
                threads.push_back(
                    std::thread(&thread_pool::worker_thread,this));    ❾
            }
        }
        catch(...)
        {
            done=true;      ❿
            throw;
        }
    }

    ~thread_pool()
    {
        done=true;          ⓫
    }

    template<typename FunctionType>
    void submit(FunctionType f)
    {
        work_queue.push(std::function<void()>(f));     ⓬
    }
};
```

This implementation has a vector of worker threads ❷ and uses one of the thread-safe queues from chapter 6 ❶ to manage the queue of work. In this case, users can't wait for the tasks, and they can't return any values, so you can use `std::function<void()>` to encapsulate your tasks. The `submit()` function then wraps whatever function or callable object is supplied inside a `std::function<void()>` instance and pushes it on the queue ⓬.

The threads are started in the constructor: you use `std::thread::hardware_concurrency()` to tell you how many concurrent threads the hardware can support ❽, and you create that many threads running your `worker_thread()` member function ❾.